

How Quorum Works

[How Quorum Works — Prototype Explanation](#)

- [1. The problem we’re solving](#)
- [2. The data \(what we’re investigating\)](#)
- [3. The KPI semantic layer — `kpi\_contracts.json`](#)
- [4. The pipeline, stage by stage](#)
- [5. Why this design, in one paragraph](#)
- [6. What’s not built yet \(be upfront about this if asked\)](#)
- [7. How to run the whole thing end-to-end](#)

How Quorum Works — Prototype Explanation

This document explains, in plain terms, what every piece of the Quorum prototype does and why it exists. Read this before your pitch so you can explain any part of it on the spot if a judge asks.

1. The problem we’re solving

A dashboard can tell you revenue dropped 8% in a region. It can’t tell you **why**, and the “why” usually lives in four places that never talk to each other: the sales system, ops/change logs, customer support tickets, and the regional rep’s head. An analyst spends days manually stitching these together — or worse, someone asks a single AI to “explain the drop,” and it confidently blends four fragments of evidence into one smooth, sometimes wrong, story.

Quorum’s idea: instead of one AI guessing, run four independent “specialist” investigators who can’t see each other’s work, then have a Falsifier try to poke holes in what they found, then have an Arbiter weigh everything and give a final, honest verdict — including saying “we don’t know yet” when that’s the truth.

2. The data (what we’re investigating)

We built a synthetic scenario for BizzGrow Inc. — three regions (APAC, EMEA, NA) plus a newly-launched APAC-South — over 12 weeks, with five data sources of different types and refresh speeds:

Source	What it is	File
Sales DB	weekly revenue, price, units per region	<code>sales_weekly.csv</code>
Ops/change log	logged internal events (price changes, competitor intel, promos, launches)	<code>ops_events.csv</code>
Support/CRM tickets	unstructured customer text	<code>customer_tickets.csv</code>
CRM deals	won/lost deals with reasons	<code>deals.csv</code>

Marketing platform	weekly ad spend	marketing_spend.csv
--------------------	-----------------	---------------------

The story baked into the data: APAC revenue drops ~17-21% starting week 9. Two real causes overlap: a 6% price increase (week 8) AND a competitor (“NimbusCo”) undercutting on deals (week 9 onward). Untangling which cause explains how much of the drop is the whole point of the system.

We also deliberately built in three traps to prove the system doesn’t just pattern-match: - **EMEA** has a ticket spike, but it’s about a resolved billing outage, not price — the system must NOT fold this into the pricing story. - **NA** has 2 tickets that trip a keyword filter (mentioning “pricing” and “discount”) but are actually happy customers, not complaints. - **APAC-South** is brand new (1-4 weeks of history) — the system must admit it doesn’t have enough data for a confident historical comparison, not fake one.

We also wired in a **second, real-world dataset**: the public Kaggle “Superstore” dataset, aggregated into a Furniture-category profit-margin KPI (`ingest_superstore.py` → `superstore_snapshot_kpi.csv`). This proves the pipeline isn’t hardcoded to one scripted story — it finds genuine, unscripted anomalies (several region/sub-category combos with real negative margins) in data we didn’t design.

3. The KPI semantic layer — `kpi_contracts.json`

Before any code touches raw numbers, every KPI is defined once in a single contract: what it means, how it’s calculated, which source it comes from, how often it refreshes, what threshold makes a movement “material,” who owns it, and who’s allowed to see it (by role, and by region row-level access). Every other script reads its rules from here instead of having thresholds hardcoded — this is the “governed KPI semantics” the brief asks for, and it’s why the whole system can extend to a new KPI by adding one JSON entry rather than rewriting code.

4. The pipeline, stage by stage

Run in this exact order (each stage reads the previous stage’s output):

<code>generate_data.py</code>	-> builds the synthetic CSVs	
<code>ingest_superstore.py</code>	-> (optional) adds the real Superstore KPI	
<code>detect_movements.py</code>	-> flags material movements	[NON-LLM]
<code>statistician.py</code>	-> price/volume decomposition	[NON-LLM]
<code>chronicler.py</code>	-> joins ops log evidence	[NON-LLM]
<code>pattern.py</code>	-> historical analog matching	[NON-LLM]
<code>field.py</code>	-> summarizes customer tickets	[LLM CALL]
<code>falsifier.py</code>	-> cross-checks all four agents	[NON-LLM]
<code>arbiter.py</code>	-> final weighted verdict	[NON-LLM]
<code>access_control.py</code> / <code>persona_render.py</code> -> role-gated, persona-specific views		

4.1 `detect_movements.py` — Detection layer (non-LLM)

Reads each KPI’s `materiality_threshold` rule straight from the contract (e.g. “flag if revenue moves >8% vs a 4-week trailing average”) and checks every region-week against it. **No LLM is involved in deciding what counts as material** — this is pure arithmetic. Output: `flagged_movements.json`, the list of things worth investigating further. This is also where we caught a real, honest false positive: NA’s ticket count got flagged because two tickets happened to contain the words “pricing” and “discount,” even though both tickets are positive. That’s a deliberate demonstration of why keyword-

only detection isn't enough on its own — which is exactly what the rest of the pipeline exists to catch.

4.2 The four specialists (read `flagged_movements.json`, work in parallel/independently)

- **Statistician** (`statistician.py`, non-LLM) — for every flagged revenue movement, splits the change into a **price effect** and a **volume effect** using a standard bridge decomposition:
 $\text{price_effect} = (P1 - P0) * Q0$, $\text{volume_effect} = (Q1 - Q0) * P0$. This is real arithmetic, not an LLM's estimate. It found that APAC's price increase would have actually *added* revenue on its own — the volume effect (lost customers) explains 100-120% of the actual decline. That's the single most important number in the whole system: it numerically proves the story is about the competitor, not the price rise.
- **Chronicler** (`chronicler.py`, non-LLM) — joins each flagged region-week against the ops/change log within a trailing window, looking for a logged event that could explain it. Found the price-change and competitor-intel events for APAC, and correctly dropped the price-change event once it aged out of the relevant window in later weeks.
- **Pattern** (`pattern.py`, non-LLM) — checks whether a similar % movement has happened elsewhere in the region's own history. Explicitly returns "no reliable analog" for APAC-South rather than forcing a weak match, because it only has 1 week of trailing data — this is the sparse-history handling the brief specifically asks for.
- **Field** (`field.py`, **the one LLM call in the whole system**) — retrieves the exact tickets that triggered a complaint-volume flag (using the same keyword filter as detection, so there's no drift between what got flagged and what gets explained) and asks an LLM to summarize what customers are actually saying. Its system prompt explicitly forbids it from inventing a negative narrative just because it was asked to investigate a "complaint" flag — which is exactly what let it correctly identify the NA tickets as positive, not complaints, contradicting the keyword-based flag that triggered it. Uses OpenRouter with a free-tier model (currently defaulting to NVIDIA Nemotron Ultra, with several other free models as automatic fallback if one is slow, rate-limited, or unavailable that day).

4.3 `falsifier.py` — "Tries to disprove every claim" (non-LLM)

Cross-checks the four specialists' outputs against each other using deterministic rules: - For revenue movements: does the numerically dominant driver (price vs. volume) have a matching ops event? Does Field's ticket summary agree? Does Pattern have historical confidence? Combines these into an overall confidence tier (high/medium/low). - For complaint-volume flags: checks Field's own confidence. If Field came back LOW confidence / positive sentiment despite the flag firing, the underlying flag gets explicitly marked **REJECTED** — this is where the NA false positive officially gets caught and killed before it reaches the final verdict.

4.4 `arbiter.py` — Final verdict (non-LLM)

Groups individual region-weeks into full "episodes" (APAC weeks 9-12 is one continuous story, not four separate ones), computes a revenue-weighted attribution split across the episode, and writes the final natural-language verdict — e.g.:

APAC: revenue down \$289,003 across weeks 9-12. Price effect: -14% | Volume/competitive effect: +111% (dominant driver). Confidence: MEDIUM. Deal-level check: 10/12 lost deals, 5 cite the competitor, 8 cite price (5 cite both). Recommendation: continue monitoring / review win-loss reasons on the competitor-cited deals.

Honest limitation, stated directly in the code: "weighing by past accuracy" (the Arbiter's

defining idea) currently uses a fixed set of prior trust weights (numeric sources trusted most, thin pattern-matching trusted least) rather than weights *learned* from real analyst override history, because that history doesn't exist yet in a fresh prototype. This is a legitimate, honestly-stated roadmap item — building the feedback loop that lets Arbiter's weights improve over time as analysts confirm or override verdicts.

4.5 `access_control.py` + `persona_render.py` — Governance and personas

`access_control.py` enforces the row-level region scoping and role-permission rules already declared in `kpi_contracts.json` — a regional sales manager scoped to EMEA gets denied when requesting APAC data, and every allow/deny decision is written to an append-only `access_audit_log.json`. `persona_render.py` takes the same Arbiter verdict and renders it two different ways depending on who's asking: a CFO gets a 3-line financial-impact summary with one action; a regional sales manager gets full deal-level tactical detail — proving the system can serve different audiences from one underlying source of truth, gated through real access control rather than just cosmetic filtering.

4.6 Telemetry

`field.py` logs latency, token usage, and estimated cost for every LLM call it makes, written to `telemetry.json`. On a real run: 2 calls, ~12.6 seconds total, 1,523 tokens, \$0.00 (free-tier model). This directly answers the brief's requirement to show LLM economics — token consumption, latency, and cost per insight — with real measured numbers, not estimates.

5. Why this design, in one paragraph

The brief explicitly warns against treating an LLM as the source of quantitative truth. Quorum's answer: **only one of nine pipeline stages calls an LLM at all**, and even that one stage (Field) is deliberately scoped to something LLMs are actually good at — reading free text and summarizing sentiment — while every number, threshold, join, and attribution calculation in the system is plain, auditable Python. When the system is uncertain (APAC-South's thin history, NA's rejected false positive), it says so explicitly instead of papering over the gap with a confident-sounding sentence.

6. What's not built yet (be upfront about this if asked)

- **Real feedback loop:** Arbiter's agent-trust weights are static priors, not learned from actual analyst override history yet.
- **UI:** everything currently runs from the command line; a "panel debate" visual dashboard showing each agent's independent claim before the verdict would be a strong addition before the pitch.
- **Live production data connectors:** current sources are CSVs (synthetic
 - one real public dataset), not live Snowflake/Databricks/Tableau connections — reasonable for a prototype per the brief's own instructions not to expect real enterprise data access.
- **Multi-KPI cross-correlation at scale:** currently reasons about each flagged movement fairly independently; a more mature version would model correlated movements across many KPIs simultaneously.

7. How to run the whole thing end-to-end

```
python generate_data.py
python ingest_superstore.py --input .\SampleSuperstore.csv --outdir .
python detect_movements.py --datadir .
python statistician.py --datadir .
python chronicler.py --datadir .
python pattern.py --datadir .
python field.py --datadir .
python falsifier.py --datadir .
python arbiter.py --datadir .
python access_control.py --datadir .
python persona_render.py --datadir . --user cfo_maria
python persona_render.py --datadir . --user sales_mgr_raj_apac
```